

Тема урока: методы экземпляра класса

1. Примеры проектирования классов

Аннотация. Урок посвящен проектированию классов.

Примеры проектирования классов

Перед тем как вы приступите к решению задач, предлагаем вместе реализовать небольшой класс `PiggyBank`, описывающий копилку. Класс будем реализовывать постепенно, шаг за шагом, развивая его функционал.

Главной характеристикой любой копилки является количество хранящихся внутри нее монет, поэтому первое, что мы сделаем, определим метод `__init__()`, который будет принимать один аргумент — начальное количество монет внутри копилки:

```
class PiggyBank:
    def __init__(self, balance):
        self.balance = balance
```

количество монет в копилке

Так как создаваемые копилки изначально, скорее всего, будут пустыми, параметру `balance` можно установить значение по умолчанию, равное `0`:

```
class PiggyBank:
    def __init__(self, balance=0):
        self.balance = balance
```

Логичным продолжением будет реализация методов, позволяющих добавлять монету в копилку а также вынимать монету из копилки. Назовем эти методы `add_coins()` и `remove_coins()` соответственно:

```
class PiggyBank:
    def __init__(self, balance=0):
        self.balance = balance

    def add_coins(self):
        self.balance += 1
монет на единицу                                # увеличиваем количество

    def remove_coins(self):
        self.balance -= 1
на единицу                                     # уменьшаем количество монет
```

Тут же мы можем разрешить методам `add_coins()` и `remove_coins()` принимать один аргумент — количество добавляемых/вынимаемых монет.

```
class PiggyBank:
    def __init__(self, balance=0):
        self.balance = balance

    def add_coins(self, coins):
        self.balance += coins

    def remove_coins(self, coins):
        self.balance -= coins
```

На данном этапе мы уже имеем неплохой функционал: мы можем отслеживать состояние монет, а также увеличивать и уменьшать их количество.

Приведенный ниже код:

```
class PiggyBank:
    def __init__(self, balance=0):
        self.balance = balance

    def add_coins(self, coins):
        self.balance += coins

    def remove_coins(self, coins):
        self.balance -= coins

piggybank = PiggyBank()

print(piggybank.balance)                # по умолчанию баланс равен
нулю

piggybank.add_coins(4)                   # добавляем 4 монеты
piggybank.add_coins(3)                   # добавляем 3 монеты

print(piggybank.balance)

piggybank.remove_coins(2)                 # вынимаем 2 монеты
piggybank.remove_coins(1)                 # вынимаем 1 монету
piggybank.remove_coins(1)                 # вынимаем 1 монету

print(piggybank.balance)
```

ВЫВОДИТ:

```
0
7
3
```

Однако несложно заметить, что у такой реализации имеется ряд проблем: мы можем бесконечно пополнять копилку, а также вынимать из нее любое количество монет, в результате чего сделать баланс отрицательным.

Начнем с решения первой проблемы. Добавим методу `__init__()` дополнительный параметр `volume` со значением по умолчанию `400`, отвечающий за максимальное количество монет, которое может содержаться в копилке. Также модифицируем метод `add_coins()`, который не будет добавлять монеты в копилку, если ее вместимость не позволяет этого сделать:

```

class PiggyBank:
    def __init__(self, balance=0, volume=400):
        self.balance = balance
        self.volume = volume                # вместимость копилки

    def add_coins(self, coins):
        if self.balance + coins > self.volume:    # проверка на возможность
добавить монеты
            print('Копилка слишком мала')
        else:
            self.balance += coins

    def remove_coins(self, coins):
        self.balance -= coins

```

Аналогично решим вторую проблему, модифицировав метод `remove_coins()`, который не будет вынимать монеты из копилки, если их в ней недостаточно:

```

class PiggyBank:
    def __init__(self, balance=0, volume=400):
        self.balance = balance
        self.volume = volume

    def add_coins(self, coins):
        if self.balance + coins > self.volume:
            print('Копилка слишком мала')
        else:
            self.balance += coins

    def remove_coins(self, coins):
        if self.balance - coins < 0:
            print('В копилке недостаточно монет')
        else:
            self.balance -= coins

```

Теперь проверим работу модифицированных методов на практике.

Приведенный ниже код:

```
class PiggyBank:
    def __init__(self, balance=0, volume=400):
        self.balance = balance
        self.volume = volume
```

